

RESEARCH

Open Access



Empirical insights into the influence of ChatGPT-facilitated programming on primary learners' performances: a mixed methods approach

Jie Xu¹, Dan Sun^{2*}, Yue Feng³, Rustam Shadiey¹ and Yan Li^{1*}

Abstract

Background ChatGPT, an AI-driven chatbot capable of generating code and providing timely, personalized feedback, offers new opportunities for supporting programming learning among younger learners. This study employed a quasi-experimental design to examine the effects of Self-Directed Programming (SDP) and ChatGPT-Facilitated Programming (CFP) learning modes on primary learners' programming knowledge, skills, and attitudes. In addition, learners' programming behaviors were explored within the CFP learning mode. A total of 71 primary learners were randomly assigned to either the CFP or SDP class. Learners in the SDP class engaged in self-directed programming activities, whereas learners in the CFP class learned programming through a ChatGPT-facilitated learning platform, CodewithAI. A mixed-methods approach was adopted to collect and analyze multidimensional data.

Results The results revealed a significant difference in programming knowledge performance between the CFP and SDP classes. Behavioral analysis indicated that primary learners in the CFP class frequently sought assistance from ChatGPT when encountering programming challenges, particularly by requesting explanations of code and error messages. In addition, primary learners in the CFP class reported relatively high levels of confidence compared with those in the SDP class.

Conclusion Based on these findings, this study offers implications for the design of primary programming instruction and the development of AI-supported instructional tools. By providing fine-grained empirical evidence, this study contributes to the field of the effectiveness of ChatGPT-facilitated programming learning for primary students.

Keywords ChatGPT, ChatGPT-facilitated programming, Mixed methods approach, Behavioral analysis, Primary school

*Correspondence:

Dan Sun
dansun@hznu.edu.cn
Yan Li
yanli@zju.edu.cn

¹College of Education, Zhejiang University, Hangzhou, China

²Chinese Education Modernization Research Institute, Hangzhou Normal University, Hangzhou, China

³Hangzhou Sandun Primary School, Hangzhou, China

© The Author(s) 2026. **Open Access** This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

Introduction

Programming education plays a critical role in fostering young learners' computational thinking and problem-solving abilities (Kong & Wang, 2021; Topali & Mikropoulos, 2021). As the importance of these competencies has become increasingly recognized, programming has been incorporated as a compulsory subject in primary education across many countries (Heintz et al., 2016; Ou et al., 2023). In China, this trend was further reinforced in 2022 with the release of the Information Science and Technology Curriculum Standards for compulsory education, which mandate the inclusion of programming instruction in Grades 5 and 6 (Ministry of Education, 2022). In traditional programming instruction, teachers typically introduce programming concepts through verbal explanations, after which learners complete tasks in a self-directed manner (Sun et al., 2021). However, for learners in non-English speaking contexts, the specialized vocabulary of programming and the abstract nature of programming concepts can pose substantial challenges to comprehension (Kwon, 2017; Shi et al., 2018). Moreover, programming instructors may face challenges such as limited professional training, insufficient time to design high-quality programming activities, and difficulties providing timely, individualized feedback to students due to large class sizes (Gomes et al., 2017; Sentance & Csizmadia, 2017; Ye et al., 2025).

The emergence of ChatGPT has introduced new possibilities for education practice by offering efficient and accessible learning support through natural language interaction and text generation (Bringula, 2024). Notably, ChatGPT's code generation capabilities enable it to assist learners and programmers with a range of programming-related tasks, including code completion, text-to-code generation, and debugging (Hu et al., 2023; Liao et al., 2024). Prior research has shown that real-time feedback from ChatGPT can provide personalized support for students during programming learning (Sun et al. 2024a), helping them address programming-related questions and access the conceptual explanations they need in a timely manner. Moreover, emerging evidence suggests that ChatGPT-facilitated programming may simplify the understanding of abstract programming concepts, reduce learners' cognitive load, foster the development of computational thinking, and increase engagement, motivation, and interest in programming activities (Haindl & Weinberger, 2024; Liao et al., 2024; Ye et al., 2025). Increasingly, researchers have examined ChatGPT-supported programming learning in K-12 contexts. For example, Khor et al. (2025) employed an extended Technology Acceptance Model (TAM) to investigate students' perceptions of AI-generated feedback in programming learning. Khor & Chan (2025) explored the effects of generative-AI-based scaffolding strategies on programming

learning using a single-group design. Despite these emerging efforts, challenges remain in integrating ChatGPT directly into the learning process. These include its potential to generate inaccurate or fabricated information (Sun et al. 2024a), foster learners' overreliance on technology, and potentially decrease critical thinking (Guner et al., 2024; Manorat et al., 2025; Uğraş et al., 2024). Consequently, the role and impact of ChatGPT in K-12 programming education deserve further empirical investigation.

In addition, relatively few studies examined K-12 learners' participation in programming practice from a process-oriented perspective. For instance, Ye et al. (2025) investigated the effects of ChatGPT on middle school students' programming performance, computational thinking, and programming self-efficacy. However, their analysis was primarily outcome-focused and provided limited insight into the behavioral processes through which learners interacted with ChatGPT during programming activities. In particular, how learners interact with ChatGPT to complete programming work is worth exploring (Sun et al. 2024a; Ouyang et al. 2024). Recent work has highlighted the importance of examining programming learning as a dynamic process. For instance, Gao et al. (2025) traced distinct learning trajectories using engagement and code metrics, revealing heterogeneous patterns among novice college learners. By adopting a process-oriented perspective to analyze learners' cognitive and metacognitive behaviors during programming learning, researchers can gain deeper insight into the progression of programming activities (Pereira et al., 2020; Wu et al., 2019). By paying attention to these details, educators can more effectively leverage ChatGPT to support and enhance young learners' programming learning experiences, thereby promoting deeper engagement and more meaningful understanding.

To address this research gap, the present study designed and developed a ChatGPT-facilitated programming learning platform, CodewithAI, tailored for primary learners. A quasi-experimental mixed-methods design was employed to examine the effectiveness of Self-Directed Programming (SDP) and ChatGPT-Facilitated Programming (CFP) learning modes in the context of Chinese primary education. Specifically, statistical analyses were conducted to examine the effects of two learning modes on primary learners' programming knowledge, skills, and attitudes (RQ1 and RQ3). To further capture learners' programming process, lag sequential analysis was applied to investigate behavioral patterns in the CFP class during ChatGPT-assisted programming activities (RQ2). In addition, the students' interview data and reflection texts were analyzed using thematic analysis to explore their learning attitudes, thereby complementing the quantitative findings (RQ3). The findings of this study

aim to provide educators and policymakers with empirically grounded insights, demonstrating the potential of generative AI tools to transform and enhance programming education in primary schools. Specifically, this study proposed to answer the following questions:

RQ 1: Are there any differences in learners' programming knowledge and skills between the CFP class and the SDP class?

RQ 2: What programming behavior patterns do learners exhibit in the CFP class?

RQ 3: Are there any differences in learners' programming attitudes between the CFP class and the SDP class?

Literature review

Research on K-12 programming education

Educators worldwide increasingly emphasize the integration of programming into K-12 education (Lindberg et al., 2019). Prior research and educational practice indicate that learning programming can foster key computational thinking skills, including abstraction, decomposition, logical thinking, pattern recognition, algorithm design, and debugging (Piedade et al., 2020). Introducing programming at an early stage also provides learners with foundational experiences that may support their future participation in STEM-related fields and careers (Grover et al., 2019). However, some studies have reported that learning programming poses substantial challenges for primary school students. These challenges include the abstract programming concepts, difficulties in understanding code meanings, language barriers faced by non-native English speakers, and struggles with debugging error messages (Duncan et al., 2014; Lye & Koh, 2014; Topali & Mikropoulos, 2021). Empirical evidence further indicates that K-12 learners show deficiencies across multiple dimensions of programming learning. For instance, their foundational knowledge is significantly limited, overall programming proficiency remains relatively low, and most students achieve only a moderate level of competence (Ou et al., 2023).

Grounded in constructivist theory, knowledge acquisition occurs when learners actively construct meaning as they internalize information within supported learning contexts following teachers' introduction of relevant problem situations (Papert, 1991). Given the complexity of programming learning, providing learners with targeted scaffolding and sufficient opportunities for exploration can facilitate a deeper understanding of programming concepts and promote the active construction of programming knowledge. Consequently, researchers have explored various instructional strategies to support programming learning, including game-based learning (Cruz García et al., 2021; Demirkiran & Tansu Hocanin,

2021; Wong & Cheung, 2020), task-based learning (Sun et al. 2024a), collaborative learning (Iskrenovic-Momicilovic, 2019; Ouyang et al., 2022), jigsaw collaborative learning method in a metaverse virtual space (Chen et al., 2024), and unplugged programming approaches (Sun et al. 2024c; Wong 2024). These approaches have been shown to enhance learners' understanding of programming concepts, as well as their satisfaction, motivation, and development of computational thinking. However, prior research indicates that primary school students often lack access to personalized learning support. In many classroom settings, large class sizes or group sizes constrain teachers' capacity to provide real-time feedback and support to each learner (Becker et al., 2016; Neutens et al., 2021). In this context, emerging AI technologies, especially Large Language Models (LLMs), offer promising opportunities to address these limitations. By delivering immediate, adaptive feedback and on-demand guidance during programming activities, such technologies have the potential to support learners' understanding and skill development more effectively (Jing et al., 2024).

Integrating ChatGPT in the programming field

ChatGPT has garnered considerable attention in education, particularly in programming education, due to its capabilities for automatic code generation, debugging, and grading (Liao et al., 2024). Unlike intelligent tutoring systems (ITSs), ChatGPT, as an AI language model, can communicate with users using natural language and provide personalized learning support, making it accessible even to learners without programming experience. However, most current ITSs are heuristic-based, i.e., they follow hard-coded rules to determine which action to provide for a given student state (Mousavinasab et al., 2021). The heuristic-based approach has drawbacks, including the need for human expertise, limited generalization, and an inability to provide personalized actions (Vujinović et al., 2024). Research indicates that ChatGPT offers exciting opportunities for students, including customized feedback, increased accessibility, interactive conversations, evaluation, and new ways to teach complex concepts (Rahman & Watanobe, 2023). Haindl and Weinberger (2024) demonstrated that ChatGPT could provide immediate feedback on students' coding problems and offer interactive support, exhibiting capabilities comparable to those of human teachers across various programming education tasks. Similarly, Shoufan (2023) reported that students admired ChatGPT's capabilities, finding it interesting, inspiring, and helpful for both learning and work. Participants found ChatGPT easy to use and appreciated its user-friendly interface, as well as the clarity and structure of its responses and explanations. Collectively, these findings suggest that ChatGPT has the potential to function as a personalized

learning companion, offering sustained support for primary school students' programming learning (Bringula, 2024; Dunder et al., 2024). Furthermore, research has found that ChatGPT can enhance students' understanding of programming concepts, computational thinking, and learning attitudes (Liao et al., 2024). Most students perceive ChatGPT positively, considering it effective for learning programming concepts, acquiring background knowledge, understanding syntax, and generating appropriate algorithms (Haindl & Weinberger, 2024). Sun et al. (2024a) further explored the potential impacts of ChatGPT-facilitated programming modes on university students' programming performance and cognition. The results indicated that such modes not only improved programming skills but also significantly influenced students' perceived usefulness, perceived ease of use, and willingness to use ChatGPT tools. Fan et al. (2025) found that compared to individual programming, AI-assisted pair programming significantly increased intrinsic motivation and reduced programming anxiety. In the broader context of programming education, these studies emphasize that ChatGPT offers more than problem-solving capabilities. It provided students with valuable resources to address programming challenges and explore various problem-solving methods, thereby increasing their interest and motivation.

Despite ChatGPT's several advantages, its application in programming education also presents challenges. For example, ChatGPT may generate incorrect or misleading information (Bringula, 2024; Uğraş et al., 2024), and its overuse may negatively impact students' critical thinking and problem-solving abilities (Michel-Villarreal et al., 2023; Ribeiro et al., 2023). To address these challenges for young learners, it is essential to redesign the interfaces of generative AI tools (e.g., adding triggers), adapt their functionalities to align with learners' educational needs, and develop more learner-friendly dialogue systems (Ma et al., 2025). For instance, Han and Cai (2023) integrated ChatGPT into an educational application to support children's literacy skills by incorporating personalized features and gamified elements. Considering both the opportunities and challenges that ChatGPT presents for programming education, further empirical research is needed to systematically examine its applications in K-12 contexts (Munaye et al., 2025). In response to this need, we designed and developed a ChatGPT-facilitated programming learning platform, CodewithAI, specifically tailored to the learning characteristics of young learners.

Although related research suggests that ChatGPT can assist learners in programming, relatively few studies have examined learners' interaction behaviors during ChatGPT-assisted programming. Most studies focus on summarizing and evaluating learners' acquisition of programming knowledge, improvement in computational

thinking skills, or self-reported perceptions of programming. For example, Shoufan (2023) asked students to complete programming activities and then reflect on their experiences with ChatGPT in their own words; these reflections were analyzed using coding and thematic analysis to examine students' perception of the tool. Hu et al. (2023) conducted a quasi-experimental study in which students were assigned to either a ChatGPT-supported group or a control group to investigate the impact of ChatGPT on programming ability and learning outcomes. Their findings showed that ChatGPT provided students with opportunities to learn programming and positively supported their programming learning. Similarly, Liao et al. (2024) adopted a quasi-experimental study combining questionnaires and interviews to examine the usability of ChatGPT as a scaffold for programming learning, and their results showed that ChatGPT effectively enhanced learners' computational thinking. Overall, existing studies have primarily assessed the impact of ChatGPT on learners' skills and learning outcomes through quantitative methods, with relatively limited integration of qualitative research. Moreover, qualitative investigations relied primarily on interviews or self-reported data, paying little attention to learners' actual behavioral interactions with ChatGPT during programming activities. This limitation makes it difficult to accurately understand the impact of ChatGPT on learners' programming behavior. Exploring learners' use of ChatGPT from a mixed-methods perspective, integrating quantitative outcomes with qualitative and behavioral analyses, can therefore provide more comprehensive insights into the role of generative AI in programming education.

To explore the effectiveness of ChatGPT in supporting programming learning and enhancing the quality of programming education, this study implemented two learning modes in Chinese primary education: Self-Directed Programming (SDP) and ChatGPT-Facilitated Programming (CFP). To support the CFP mode, the CodewithAI platform was developed by integrating ChatGPT with adjustments tailored for primary learners, such as simplified interaction through preset buttons and constraints that ensure feedback remains focused on programming-related tasks. This study employed a quasi-experimental design to compare the effects of two learning modes on novice programmers, thereby providing empirical evidence for the design of programming education. Both summative and process-oriented perspectives were incorporated. From a summative perspective, the study examined changes in learners' programming knowledge, skills, and attitudes before and after participation in the two learning modes. For a process-oriented perspective, learners' programming behaviors in the CFP mode were analyzed in detail. A mixed-methods approach was

employed, combining statistical analyses of knowledge, skills, and attitudes measures with qualitative analyses of interview and reflection text data and sequential analyses of programming behavior data. Together, these analyses offer a comprehensive understanding of learners' programming experiences and provide insights into the integration of AI tools in primary programming education.

Methodology

Research context and participants

The study was conducted within a compulsory course titled *Information Science and Technology* at a public primary school in eastern China during the fall semester of 2024. According to the *Information Science and Technology Curriculum Standards for Compulsory Education* issued by the Ministry of Education in China (2022), the content was required to be taught using Python. A quasi-experimental approach was employed to investigate learners' programming knowledge, skills, and attitudes in the control class (SDP) and the experimental class (CFP). In addition, the learner's programming behavioral patterns were analyzed within the CFP condition.

Participants were students from two randomly selected sixth-grade classes, aged between 11 and 13 years. The SDP class consisted of 35 learners (20 male, 15 female), and the CFP class comprised 36 learners (19 male, 17 female). The two classes were randomly assigned to either the SDP or CFP condition. Both classes received instruction in a computer-equipped classroom, with each learner having access to an individual computer. To minimize expectancy effects, participants were not informed about the specific instructional conditions assigned to their classes. The participants were all novice programmers with no prior experience in Python. Upon securing written consent from the review committee, the data

were collected and analyzed, ensuring that no ethical infractions occurred.

Diagram of analysis

The research presented a diagram for analyzing differences in learners' programming knowledge and skills, as well as their attitudes toward programming, between the SDP and CFP classes, incorporating both summative and process assessments (see Fig. 1). For the summative assessment, researchers collected data from both classes. Programming knowledge was evaluated through performance data, including pre- and post-test results and final programming projects. Additionally, as detailed in Appendices A and B, learners' attitudes (including data from pre- and post-surveys and interviews) provide valuable insights into learners' perceptions of programming. In terms of process assessment, researchers collected behavioral data on the CFP class, including log files that record timestamps for a learner's programming actions (e.g., coding, saving, and submitting) on the platform. This dataset was rigorously analyzed to investigate programming behaviors emerging from learners' interactions in the CodewithAI platform. Overall, this analysis diagram serves as a comprehensive guide for data collection in programming education.

Instructional procedures and platform

The *Information Science and Technology* course was conducted over 10 weeks, with one 45-minute session each week. Two sixth-grade classes participated in the study and were taught by the same instructor, who had five years of experience teaching programming and had demonstrated mastery of programming knowledge and skills. To ensure instructional consistency, identical teaching materials and lesson plans were used across both classes. Under the guidance and support of the research team, the

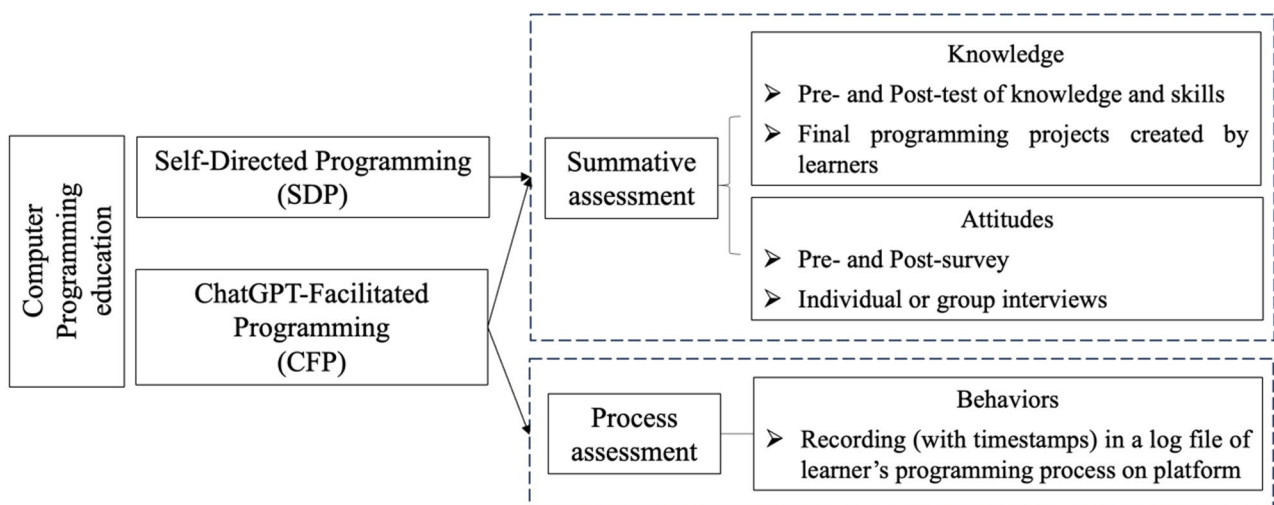


Fig. 1 Diagram of analysis in programming education

instructor structured the course into three phases. The pre-rest and post-test sessions were distributed across Phase I and Phase III, respectively. Phase II consisted of eight programming tasks, including loops, conditional statements (e.g., if, if-else), sequential statements, binary search, the enumeration method, the sieving method, and the congruence method (see Fig. 2).

Specifically, Phase I consisted of two parts (one session). In the first part, learners completed the pre-test. In the second part, the instructor surveyed the use of ChatGPT among learners in the CFP class, finding that approximately 80% of learners had not used related ChatGPT tools. The instructor then introduced the following topics: (1) a demonstration of the CodewithAI platform used in the CFP class; and (2) typical prompt frameworks and examples. For instance, the instructor introduced the CRISPE framework (CR: Capacity and Role, I: Insight, S: Statement, P: Personality, E: Experiment) (Wang et al., 2023) and then gave some examples about how to use the framework to students' real-life experiences. Here is an example: "as a teacher who was proficient in computers and wrote a blog (Capacity and Role), your readers were professionals interested in the latest advancements in artificial intelligence (Insight). Please provide examples of artificial intelligence applications in daily life (Statement). When answering the questions, please use an academic writing style (Personality). Give me a few different examples (Experiment)." After the introduction, learners became familiar with the CodewithAI platform and participated in human-AI interactions using the provided prompt framework. Similarly, learners in the SDP class completed the pre-test and then familiarized

themselves with the IDLE platform, which is the typical Integrated Development Environment (IDE) for Python. Similar to the CodewithAI platform, the IDLE platform supports writing, saving, running code, and console output (Kroustalli & Xinogalos, 2021). This means CodewithAI retains IDLE's core functionality while integrating ChatGPT.

In Phase II (eight sessions), the instructor structured every programming lesson according to the Presentation, Practice, Production (PPP) model (Liu & Chen, 2023), which aligns with constructivist theory by encouraging meaningful construction through tasks. During the Presentation part (20 min), learners attended a lecture by the instructor to grasp programming concepts and algorithms in the SDP and CFP classes. In the Practice and Production parts (25 min), learners in the SDP class completed programming tasks using IDLE. They could seek assistance from the instructor, peers, or website resources when encountering difficulties. In contrast, learners in the CFP class utilized ChatGPT to complete their programming tasks and could request assistance when needed.

In Phase III (one session), learners completed the post-test. The instructor selected five learners from each of the two classes for one-by-one interviews, which lasted 8–10 min after class and were recorded with the interviewees' permission.

The learning environment was CodewithAI, a web-based programming platform developed by the author's research team for this study (see Fig. 3). CodewithAI is a comprehensive platform divided into three distinct sections, each designed to enhance learners' experience. On

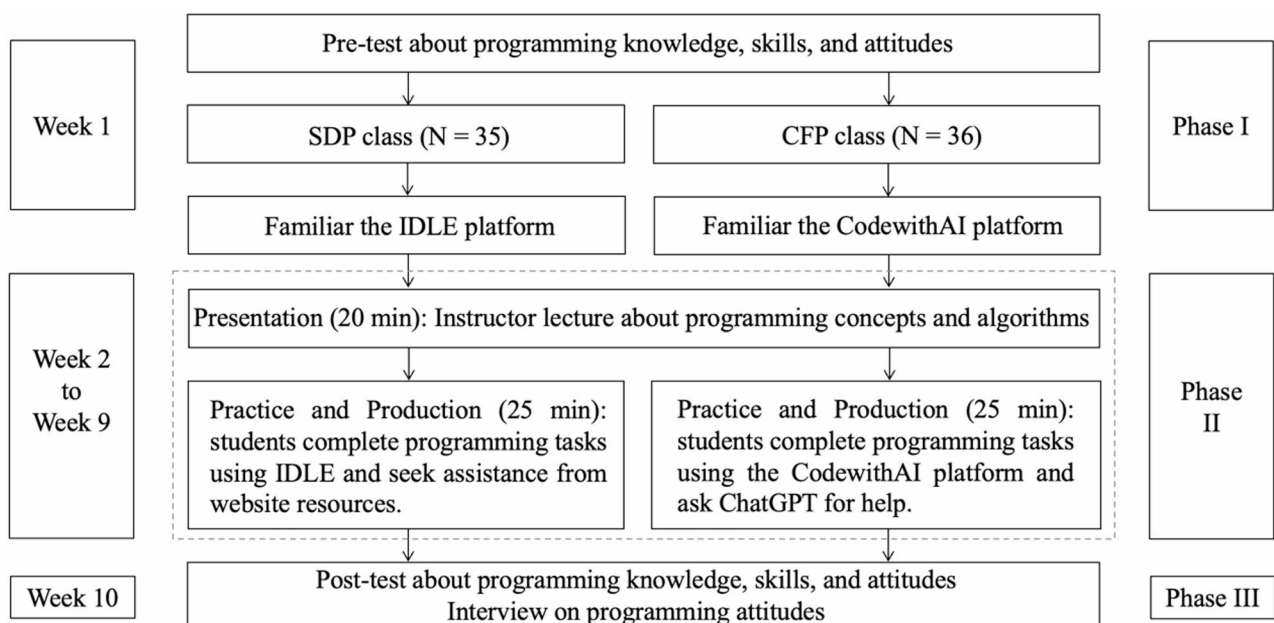


Fig. 2 Overview of experimental design and instructional process

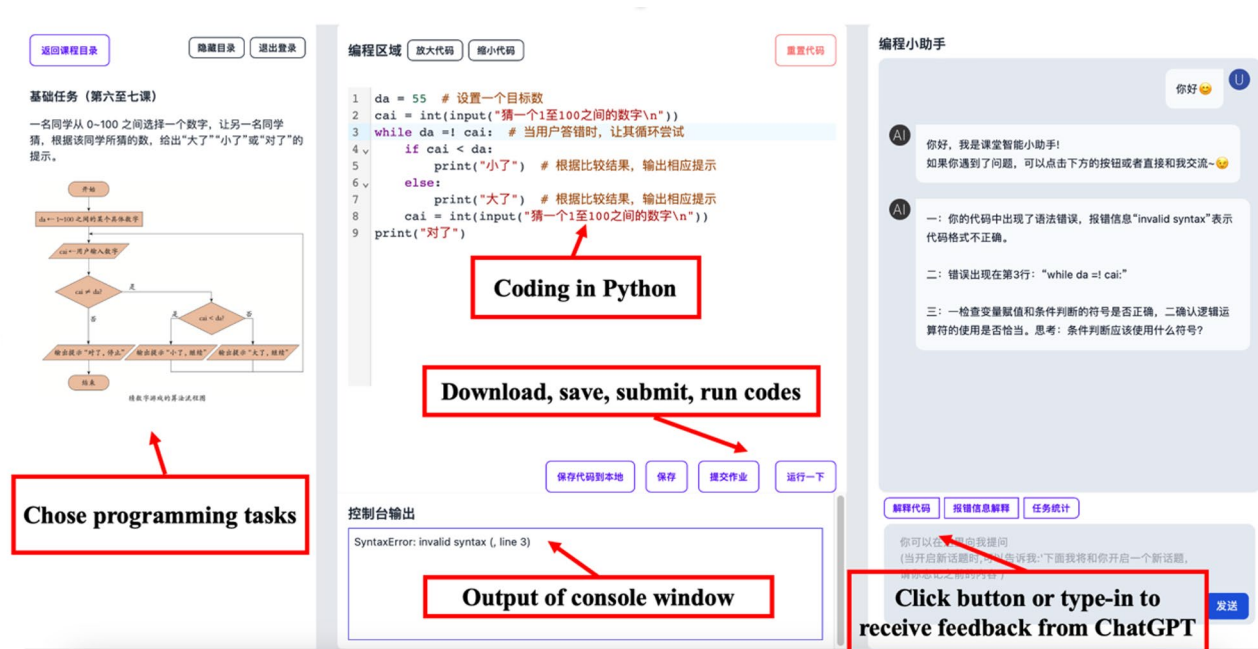


Fig. 3 The CodewithAI platform

the left, learners select Python programming tasks that have been pre-designed and sequenced by the instructor. The central section provides a Python editor and console, where learners can write, download, save, submit, and execute code. The right section integrates ChatGPT (gpt-3.5-turbo) as an instructional assistant.

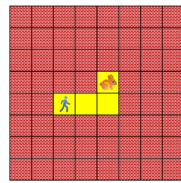
To make the platform easier and more convenient for primary school students to use and to prevent the potential disadvantages of ChatGPT (e.g., undermining learners' ability to think independently), the platform was carefully designed in the study. Unlike using the standard ChatGPT interface directly, CodewithAI integrates the LLM into a constrained, task-focused workspace tailored to primary learners. First, it is important to note that, given the randomness in content generated by LLMs, there is a possibility of receiving inconsistent or even erroneous feedback (Sun et al. 2024b). To address this, the research team adjusted the prompts for the LLMs to ensure the reliability and validity of the feedback. Second, to match the cognitive level of primary students, the platform has simplified communication by using preset functions as buttons (e.g., "explain console output", "explain current code"), so that learners can access ChatGPT support without having to formulate complex prompts. Third, ChatGPT is configured solely to answer programming-related questions. If primary school students ask questions unrelated to programming, ChatGPT will prompt them to ask programming-related questions (e.g., "I only answer questions related to programming"). Finally, to prevent learners from directly copying and pasting ChatGPT's feedback, the platform

has implemented a copy-paste restriction, encouraging learners to consider the feedback provided by ChatGPT thoughtfully. These design features collectively aim to make LLM-based support usable and pedagogically appropriate for younger learners, while reducing the risks associated with unrestricted use of a general-purpose ChatGPT interface.

Data collection and analysis approaches

This study employed four approaches to data collection and analysis. First, pre- and post-tests were conducted to assess learners' programming knowledge and skills. The programming knowledge test consisted of 14 multiple-choice questions (each with four options) in both the pre-test and post-test (see Table 1). Although the item format remained consistent, the content differed across the two tests, focusing on fundamental programming concepts and algorithms, including conditional statements, loops, sequential logic, and basic debugging. The items required learners to identify the correct code and select appropriate programming structures, providing an objective measure of learners' mastery of programming concepts. Each correct response was awarded 10 points, and total scores were subsequently normalized to a 100-point metric for analysis. The knowledge test was developed by three experienced programming instructors and aligned with the national curriculum standards to ensure content validity (Ministry of Education, 2022). The independent *t*-test was used to compare post-test scores on programming knowledge performance between the two learning modes.

Table 1 Some sample test questions about programming knowledge and skills

Section	Sample questions	
Programming knowledge	1. According to the picture provided by the instructor, which path should the boy follow to get to the grid where the little rabbit is? (B) A. right right down B. right right up C. right up up D. right down down	
	2. About Python, which of the following programs can output “hello world”? (A) (A) print (“hello world”) (B) print “hello world” C. printf (“hello world”) D. puts “hello world”	
	Dimensions	Number of items
Programming skills	Abstraction	4
	Decomposition	3
	Algorithmic thinking	4
	Evaluation	4
	Generalization	4
	Sample item	
Programming skills	I usually think of a problem from a whole point of view, rather than looking at the details.	
	I usually think if it is possible to decompose a problem.	
	I am used to figuring out the procedures step-by-step for a solution.	
	I tend to find a correct solution for a problem.	
	I tend to solve a new problem according to my experience.	

To evaluate programming skills, we followed prior studies (e.g., Sun et al., 2021) and used the Computational Thinking Skill (CTS) scale as an indicator of learners’ programming-related competencies. Previous research has developed validated instruments to assess CTS (Doleck et al., 2017; Tsai et al., 2021), which are widely used due to their accessibility and suitability for educational contexts (Kang et al., 2023). Adapted from the computational thinking scale (Tsai et al., 2021), the skills measure included five dimensions (see Table 1): identify patterns and simplify complexity (abstraction), break down problems (decomposition), develop step-by-step solutions (algorithmic thinking), assess solution effectiveness (evaluation), and transfer knowledge across contexts (generalization). Notably, the CTS was validated and applied among Chinese K-12 learners by Cheng et al. (2025). The CTS survey comprised nineteen items aligned with these dimensions and was administered in both the pre-test and post-test. The Cronbach α was 0.946, indicating good reliability. Items were scored on a five-point Likert scale ranging from 1 (strongly disagree) to 5 (strongly agree), with higher scores indicating greater consistency with each statement. The independent *t*-test was used to compare post-test scores on CT skills between the two learning modes.

Second, we collected log files that recorded learners’ programming behavior on the CodewithAI platform during the CFP class. CodewithAI has its code editor (IDE). When a learner clicks a “save” button in CodewithAI, his/her programming behaviors in the IDE (e.g., code-editing, running, submitting tasks, etc.) are recorded (with timestamps) in a log file on the server. Building on an existing coding framework (Sun et al. 2024a), the current framework was modified to align with platform-specific data. Four new programming-related behaviors were introduced to provide a more comprehensive assessment

of learners’ engagement with the platform. Finally, eight programming behaviors were categorized into two types on the platform. Programming-related behaviors: number of platform operations (NoO), number of clicks on run (NoR), running in Python incorrectly (RPI), running in Python correctly (RPC); ChatGPT-related behaviors: explaining console output in ChatGPT (ECO), explaining current codes in ChatGPT (ECC), raising own questions in ChatGPT (RoQ), submitting current codes to ChatGPT (SCC) (see Table 2).

Lag-sequential analysis (LsA) has been used in recent years to identify learner behaviors in technology-supported learning environments (Hou & Wang, 2015). We employed LsA to examine the sequential relationships among features, including behavioral transition

Table 2 Coding framework of programming behaviors

Behaviors	Codes	Description
Programming-related behaviors	Number of platform operations (NoO)	The number of platform operations made by a learner (i.e., logging, saving) in all projects.
	Number of clicks on run (NoR)	The number of clicks on “run” button made by a learner.
	Running in Python incorrectly (RPI)	A learner running Python codes and received incorrect feedback.
	Running in Python correctly (RPC)	A learner running Python codes and received correct feedback.
ChatGPT-related behaviors	Explaining console output in ChatGPT (ECO)	A learner clicking “explaining console output” button to receive feedback from ChatGPT.
	Explaining current codes in ChatGPT (ECC)	A learner clicking “explaining current codes” button to receive feedback from ChatGPT.
	Raising own questions in ChatGPT (RoQ)	A learner raising their own questions to ChatGPT.
	Submitting current codes to ChatGPT (SCC)	A learner submits their current code to ChatGPT to test its correctness.

frequencies and their network visualizations. The method has already been used in studies exploring programming and robotics instruction (Hopcan et al., 2022; Sisman et al., 2022; Tan et al., 2024). The LsA involved five measures: (1) transitional frequencies, which reflect how often a specific transition occurred within a given sequence; (2) expected transitional frequency, representing the anticipated number of transitions under the null hypothesis of independence between the codes; (3) transitional probabilities, indicating the likelihood of event B following event A; (4) adjusted residuals z-scores, which denote the statistical significance of specific transitions; and (5) Yule's Q, a standardized measure ranging from -1 to $+1$ that assesses the strength of association. Yule's Q value (calculated in RStudio) was used to assess the strength of transitions between codes, with a value of 0 indicating no association (Chen et al., 2017).

Third, to assess differences in learners' attitudes toward programming before and after the experiment, a survey was administered, which is consistent with prior research (Sun et al., 2021, 2023). The questionnaire consisted of five Likert-scale questions, ranging from 1 (strongly disagree) to 5 (strongly agree), and included items such as "I enjoy programming" (see Appendix A). The Cronbach's α was 0.909, indicating good reliability and validity. Independent *t*-tests and descriptive analyses were used to assess changes in learners' attitudes (confidence, enjoyment, and future interest) toward programming across the two learning modes. In addition, to understand learners' feelings of the programming course, an open-ended question (reflective text) was included in the post-test (i.e., please describe your learning experience in the programming course, including the knowledge and skills acquired, your attitudes towards programming, and your experience with the CodewithAI platform). This question was not mandatory. Finally, a total of 22 learners from the CFP class and 21 from the SDP class responded.

After the course, five learners from each class were selected for semi-structured interviews using stratified random sampling, ensuring representation of both high- and low-achieving learners (Rekha et al., 2024). The questions in the semi-structured interviews are more targeted and detailed, enabling instructors to gain a deeper understanding of the learners' programming learning experiences. The interview includes several open-ended questions (see Appendix B). It primarily aimed to assess learners' mastery of programming knowledge and skills through their recollections of the course content, understand behavioral changes through the learners' use of CodewithAI (combined with ChatGPT)/websites, and examine their attitudes toward programming learning by analyzing changes in their interests. A three-step sequence was used to process the interview data: (1) recording the interviews, (2) converting the recordings to

texts, and (3) double-checking the final texts (Sun et al., 2021).

All reflective texts and interviews were conducted in Chinese to allow participants accurately articulate themselves to address the questions. The quotations presented in this study were directly translated into English by the first and second authors, both experienced in Chinese-English translation. Each author independently translated a portion of the transcripts, and the translations were cross-checked for consistency and accuracy. Besides, no paraphrasing was applied, and the translations preserve the original meaning of the participants' responses.

The learners' reflective texts and interview transcripts were analyzed using an inductive thematic analysis, a widely recognized qualitative research method (Braun & Clarke, 2006; Tobias et al., 2025). The thematic analysis process involved five main steps: (1) transcription and familiarization, (2) coding the data separately by two coders, (3) theme generation, (4) independent validation, and (5) iterative refinement. First, the two coders carefully read all interview transcripts and reflection texts to become familiar with the data, gain an initial understanding of participants' programming learning experiences, and identify relevant segments for further analysis. During this phase, the coders made preliminary notes on passages reflecting participants' attitudes, experiences, and perceptions of programming learning. Next, the coders independently conducted initial inductive coding across all records, generating 32 preliminary codes. These codes were then grouped into 10 potential themes. The coders met regularly to compare coding decisions, discuss discrepancies, and refine the coding scheme through negotiated agreement. Overlapping codes were merged, clarified, and revised until full agreement was reached. To ensure the rigor of the coding process, inter-rater reliability was calculated based on 30% of the data; the two coders achieved 85% agreement, exceeding the commonly accepted threshold of 80% (Guest et al., 2012). In addition, member checking and repeated discussions with researchers were conducted to enhance the credibility of the final themes (Glesne, 2016). The reflective texts, interview transcripts, and preliminary analysis results were confirmed with every participant and allowed to revise when necessary. This process was facilitated by the first and third authors and involved eight 30-minute sessions to ensure that the themes adequately reflected students' perspectives. Subsequently, all researchers reviewed all transcripts and the initially generated themes, re-examining codes and themes that required further clarification. These iterative discussions continued until consensus was reached among all researchers. As a result, three overarching themes were identified.

Table 3 Independent t-test of post-test of learners' programming knowledge and skills in two learning modes

	Dimensions	Modes	M	SD	t	p
Programming knowledge	Post-test scores	SDP	79.39	16.32	-2.11*	0.039
		CFP	86.11	9.50		
	Differences (post-test-pre-test)	SDP	6.33	25.62	-1.00	0.323
		CFP	11.51	17.27		
Programming skills	Post-test scores	SDP	3.76	0.92	-0.19	0.850
		CFP	3.80	0.56		
	Differences (post-test-pre-test)	SDP	0.22	0.62	-0.71	0.481
		CFP	0.33	0.59		

* $p < 0.05$ **Table 4** Descriptive data of average learners' programming behaviors

	NoO	NoR	RPI	RPC	ECO	ECC	RoQ	SCC
Frequency	617	2906	1762	1144	131	63	48	96
M/SD	17.14±8.34	80.72±86.74	48.94±54.90	31.78±39.51	3.64±4.74	1.75±2.32	1.33±2.39	2.67±4.31

Results

Learners' programming knowledge and skills in SDP and CFP classes

This study investigated the use of ChatGPT in primary programming education by comparing the effects of two learning modes on learners' programming knowledge and skills (see Table 3). For programming knowledge, an independent-samples t -test indicated no significant difference was found between the two learning modes at pre-test ($t = -0.46$, $p = 0.65$), suggesting comparable baseline levels. After the 10-week intervention, a statistically significant difference in post-test scores was observed between the two learning modes ($t = -2.11$, $p = 0.039$), indicating differential learning outcomes. An independent-samples t -test comparing gain scores (post-test-pre-test) revealed no statistically significant difference between the two learning modes ($t = -1.00$, $p = 0.323$). The results suggest a meaningful difference in the development of programming knowledge between the two learning modes. For programming skills, no significant difference was found between the two learning modes in the pre-test ($t = 0.41$, $p = 0.680$). Similarly, the post-test results showed no significant difference after the intervention ($t = -0.19$, $p = 0.850$). Although both classes showed improvement from pre-test to post-test, with mean gains of 0.22 for the SDP class and 0.33 for the CFP class, an independent-samples t -test on gain scores revealed no significant difference between the two learning modes ($t = -0.71$, $p = 0.481$).

Learners' programming behaviors in the CFP class

Regarding RQ2 (What programming behavior patterns do learners exhibit in the CFP class?), we collected data on learners' programming behaviors in the CFP class (see Table 4) and used lag-sequential analysis to identify differences in behavioral patterns. The study revealed that learners predominantly engaged in three main activities:

running Python code (NoR; frequency = 2,906), receiving incorrect feedback after running code (RPI; frequency = 1,762), and receiving correct feedback after running code (RPC; frequency = 1,144). This indicated a strong focus on programming-related tasks, coupled with frequent encounters with error messages in their Python code. In terms of interactions with ChatGPT, learners primarily used the tool to seek explanations for console output (ECO; frequency = 131). For example, when learners clicked the "explaining console output" button on the CodewithAI platform, they received feedback from ChatGPT. Learners also submitted their Python code to ChatGPT for correctness assessment (SCC; frequency = 96). Specifically, when learners click the submit button, they will receive ChatGPT's evaluation of the code. Additionally, they asked ChatGPT to explain their Python code by clicking the "explaining current code" button (ECC; frequency = 63). In contrast, learners were less frequent in typing questions to ChatGPT (RoQ; frequency = 48). These findings underscore the important role of ChatGPT in supporting learners' understanding and troubleshooting during the programming process, highlighting its potential to enhance the overall learning experience for young learners.

The LsA results (see Table 5; Fig. 4) revealed several high-frequency behavioral transitions and behavioral sequence diagrams among learners using ChatGPT in programming education (Sisman et al., 2022; Tan et al., 2024). Notably, learners frequently transitioned from explaining their current codes to ChatGPT to asking questions of ChatGPT, with a Yule's Q value of 0.84. Additionally, there were significant transitions from the operating platform (NoO) to explaining current codes (ECC) (Yule's $Q = 0.56$), and from explaining console output with ChatGPT to explaining current codes (ECC) (Yule's $Q = 0.51$). Moreover, there were significant transitions from receiving incorrect feedback (RPI) to

Table 5 LSA transition frequency of learners' programming behaviors in the CFP class

CFP Transition	Description	Yule's Q
ECC-> RoQ	The learner clicked the "explaining current codes" button and then asked additional questions after receiving feedback from ChatGPT.	0.84
NoO-> ECC	The learner clicked the "explaining current codes" button after logging in to the Code-withAI platform.	0.56
ECO-> ECC	The learner first clicked the "explaining console output" button and received feedback from ChatGPT, and then they clicked the "explaining current codes" button, receiving more feedback about the code.	0.51
RPI-> ECO	When errors occurred in code, learners clicked the "explaining console output" button to obtain ChatGPT's identification of the error type.	0.49
RoQ-> ECC	After learners raised their questions to ChatGPT, they clicked the "explaining current codes" button.	0.46

explaining console output (ECO) (Yule's $Q=0.49$) and from raising one's own questions in ChatGPT (RoQ) to explaining current code (ECC) (Yule's $Q=0.46$). The frequent transitions indicated that learners relied on ChatGPT to clarify errors and understand their code, thereby enhancing their programming knowledge, demonstrating ChatGPT's effectiveness as an educational tool. By providing immediate, detailed explanations, ChatGPT could help young programmers overcome obstacles and deepen their understanding of coding concepts.

Learners' programming attitudes in SDP and CFP classes

Programming attitudes were assessed across three dimensions: confidence, enjoyment, and future interest in learning programming. Before the experiment, no significant differences were observed between the two learning modes across any of these dimensions (confidence: $p=0.904$; enjoyment: $p=0.688$; future interest: $p=0.955$). Following the 10-week intervention, a significant difference in post-test confidence scores was observed between the two classes ($t = -2.15, p=0.036$). Further analysis of gain scores (post-test-pre-test) revealed a significant difference between the two learning modes ($t = -2.37, p=0.021$). These results indicate that learners in the CFP class demonstrated a greater increase in confidence compared with those in the SDP class (see Fig. 5a). No significant differences were found between the two classes in terms of enjoyment ($t = -0.54, p=0.590$) or future interest ($t = -1.60, p=0.114$) in the post-test. Compared with pre-test scores, learners in the SDP class showed a 0.01 increase in enjoyment, whereas learners in the CFP class showed a 0.22 increase (see Table 6). However, an independent-samples t -test comparing the pre-post differences indicated no statistically significant difference between the two learning modes for enjoyment ($t = -0.99, p=0.325$). Similarly, learners in the SDP class showed a 0.14 increase in future interest, while learners in the CFP class had a 0.53 increase. However, there was no significant difference ($t = -1.75, p=0.085$) in the future interest dimension between the two classes when comparing pre-post changes. Overall, although learners in the CFP class tended to report higher mean scores, no significant differences were found between the

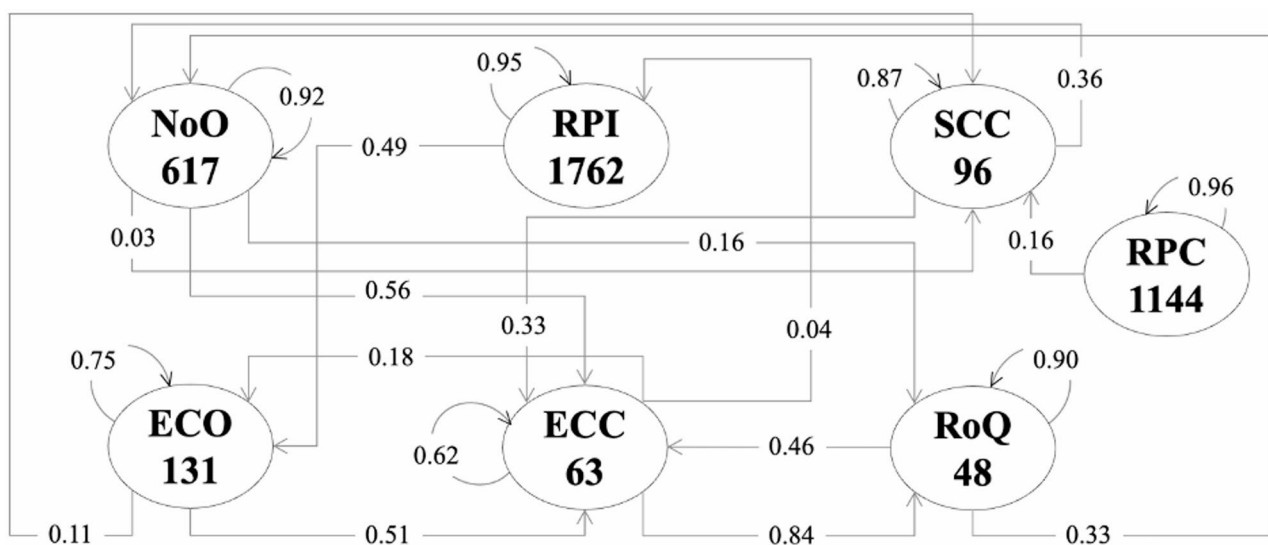


Fig. 4 Learners' behavioral sequence diagram in the CFP class. The transitional network representation of learners' behavior depicts each behavior code as a node. Yule's Q is indicated on the connecting lines to illustrate the strength of the transitional associations. NoO Number of platform operations, RPI Running in Python incorrectly, SCC Submitting current codes to ChatGPT, ECO Explaining console output in ChatGPT, ECC Explaining current codes in ChatGPT, RoQ Raising own questions in ChatGPT, RPC Running in Python correctly.

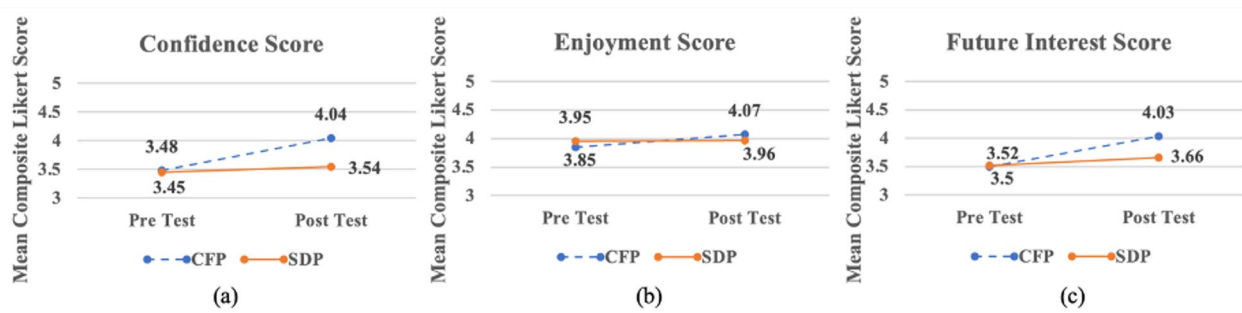


Fig. 5 Learners' score for confidence **a**, enjoyment **b** and future interest **c** in two learning modes

Table 6 Independent t-test of post-test of learners' programming attitudes in two learning modes

	Dimensions	Modes	M	SD	t	p
Confidence	Post-test scores	SDP	3.54	1.20	-2.15*	0.036
		CFP	4.04	0.67		
	Differences (post-test-pre-test)	SDP	0.09	0.78	-2.37*	0.021
		CFP	0.56	0.88		
Enjoyment	Post-test scores	SDP	3.96	1.05	-0.54	0.590
		CFP	4.07	0.63		
	Differences (post-test-pre-test)	SDP	0.01	0.83	-0.99	0.325
		CFP	0.22	0.94		
Future interest	Post-test scores	SDP	3.66	1.11	-1.60	0.114
		CFP	4.03	0.81		
	Differences (post-test-pre-test)	SDP	0.14	0.88	-1.75	0.085
		CFP	0.53	0.97		

* $p < 0.05$

two learning modes in terms of enjoyment and future interest (see Fig. 5b and c).

To complement the quantitative findings, the learners' reflections and interview data were systematically analyzed to uncover deeper insights into their programming learning experiences in the SDP and CFP classes. Two coders conducted thematic analysis, resulting in three themes (see Table 7): recall of programming knowledge and skills, programming attitudes, and experiences of programming learning.

Theme 1: Recall of programming knowledge and skills

The first theme highlighted differences in how learners from the two classes perceived and described their acquisition of programming knowledge and skills. Learners ($N=23$) in the CFP class generally demonstrated a stronger ability to accurately describe programming knowledge, whereas half of the learners in the SDP class more often provided less precise accounts of what they had learned. This suggests that learners in the CFP class tend to present a more structured understanding of programming concepts. For instance, one learner from the CFP class reported:

I can recall most of the course content, including enumeration and loops..... What impressed me the

most was the "guess the number" activity. It was very engaging and helped me quickly remember how loop statements work. (S3 from the CFP class, interview)

Most learners reported that they were able to use programming tools correctly, including IDLE in the SDP class ($N=21$) and CodewithAI in the CFP class ($N=23$). Especially for novice learners, the relative simplicity and ease of use of the Python platforms appeared to reduce cognitive load, allowing students to focus more on understanding core programming concepts. However, learners' experiences with these tools differed. Some SDP learners expressed frustration when encountering errors during coding. In contrast, CFP learners described a developmental trajectory from initial unfamiliarity to increasing proficiency when using the CodewithAI platform:

I'm pleased I learned to use IDLE, which lets me write and run code. However, sometimes the code I write produces errors, and that really frustrates me. (S13 from the SDP class, reflection text)

At first, I felt very unfamiliar with the CodewithAI platform because it was my first time learning Python, and I couldn't understand the code at all. But now, I can use the CodewithAI platform proficiently. (S1 from the CFP class, interview)

Table 7 Themes from learners' reflection and semi-interviews in two learning modes

Thematic category	Codes	Representative quotes	Frequency	
			SDP (N=26)	CFP (N=27)
Recall of programming knowledge and skills	Write down or speak the programming knowledge learned correctly	I have learned to use "if-else" conditional statements and "while" loop structures. (S2 from the CFP class)	13	23
	Learn to use programming tools correctly	I can use IDLE to open Python files and run the code. (S5 from the SDP class)	21	23
	Confusion about course contents	I can't distinguish between the congruence method and the sieving method. I don't know when to use which method. (S3 from the SDP class)	6	4
Programming attitudes	Enjoy the programming course	I enjoy the programming course, and I also study programming after class. (S2 from the CFP class)	13	17
	Confidence about other related subjects, such as mathematics	Learning programming has given me confidence in solving mathematical problems. (S10 from the SDP class).	2	8
	Interest in future programming course	I hope to continue learning programming with ChatGPT's assistance in the future. (S8 from the CFP class).	9	15
Experiences of programming learning	The CodewithAI platform enhances learning efficacy but need to improve.	ChatGPT patiently explained programming concepts, enabling me to complete the tasks quickly. While I hope it can run more smoothly when asking questions. (S4 from the CFP class).	0	15
	Improve problem-solving ability	Addressing programming problems (e.g., buying tickets) helps me learn a lot and improves my problem-solving skills. (S1 from the CFP class).	4	8
	Receive feedback timely	Receiving quick responses from ChatGPT during class keeps me motivated. (S6 from the CFP class).	6	20

Despite these gains, learners in both classes reported confusion about specific course content (SDP: $N=6$; CFP: $N=4$), particularly regarding more abstract programming concepts, such as the congruence method. In addition, limited opportunities to consolidate knowledge after class further constrained their understanding, as students reported in interviews and reflection texts:

I found it very hard to understand the congruence method in class, which affected my mood all day. (S12 from the SDP class, reflection text).

I have some impressions of what we learned in this course, but I did not fully grasp the rules and methods taught in class because I had no opportunity to consolidate them after class. (S4 from CFP class, interview)

Theme 2: Programming attitudes

The second theme revealed learners' attitudes toward programming across the two learning modes. Reflection texts and interview data indicated that learners in both classes expressed enjoyment of programming (SDP: $N=13$; CFP: $N=17$); however, the sources of their positive attitudes differed. Learners in the CFP class attributed their enjoyment to the learning experience itself, particularly their interactions with the CodewithAI platform and the sense of achievement derived from solving programming problems. In contrast, learners in the SDP class tended to associate their enjoyment with external factors. For example, one learner expressed enjoyment

primarily because of the freedom to use the computer after completing assigned tasks:

I like programming, especially solving "Han Xin's Counting Soldiers" problem. I learned the enumeration method, the sieving method, and the congruence method, all of which left a strong impression on me. (S1 from the CFP class, interview).

I enjoy the programming course because I can interact with the CodewithAI platform. (S7 from the CFP class, reflection text).

In the programming class, I'm allowed to play on the computer freely once I finish my tasks, which I really enjoy. (S5 from the SDP class, interview)

A minority of learners reported that programming activities enhanced their confidence in other related subjects, particularly mathematics. When encountering similar problems (e.g., chickens and rabbits in the same cage) in both programming and math classes, learners were able to transfer problem-solving strategies across domains. This indicates that programming learning may reinforce cross-disciplinary connections and understanding:

I learned the "chickens and rabbits in the same cage" problem during the course, which I had also encountered in my mathematics class. I find the two to be similar, and this connection helps me solve many mathematical problems. (S5 from the CFP class, interview)

When asked about their future interest in continuing programming learning, only a small number of learners in the SDP class expressed such intentions. In contrast, approximately half of the learners in the CFP class reported a desire to continue learning programming. These responses indicate that timely support and scaffolding may play a critical role in sustaining learners' motivation and future engagement in programming:

I would like to continue learning programming in the future because I find the course challenging, but I hope to receive timely assistance whenever I encounter difficulties. (S3 from the SDP class, interview)
My interests have changed; I feel more interested in learning programming. However, I felt it was difficult for me before that. (S5 from the CFP class, interview)

Theme 3: Experiences of programming learning

The third theme captured learners' experiences of programming learning in both classes. Reflection texts and interview data indicated that many learners ($N=15$) in the CFP class perceived the CodewithAI platform as enhancing their learning efficacy. While these learners reported that they could acquire programming knowledge more easily when encountering difficulties, they also expressed dissatisfaction with the CodewithAI platform, citing issues such as unstable performance, lengthy responses, the absence of voice interaction, difficulty identifying key points, and small font sizes:

The CodewithAI platform is easy to understand. Although the code is challenging, I can interact with ChatGPT to complete it, which has enhanced my learning efficacy. (S4 from the CFP class, interview)
When using the CodewithAI platform, if I encounter code I don't understand or error codes, ChatGPT can immediately explain their meanings, helping me understand them better. Then I can finish the task quickly. However, the answers from the CodewithAI platform are too long, making it hard for me to grasp the key points. (S11 from the CFP class, reflection text)
When I have multiple questions, typing questions is time-consuming. I hope the CodewithAI platform could provide a voice input function. (S13 from the CFP class, reflection text)

Interestingly, several learners ($N=8$) in both classes perceived gains in their problem-solving abilities through programming activities. In the SDP class, four learners reported that solving programming problems (e.g., buying tickets, guessing the number) improved their problem-solving abilities, and eight learners from the

CFP class noted that resolving programming challenges clarified their logical thinking and enabled them to break down problems step by step when facing difficulties. The interview results were consistent with the learners' reflection texts:

During the programming course, what impressed me most was the idea of problem decomposition: breaking a big problem into smaller ones and then forming solutions for each. I feel that my problem-solving skills have improved. (S2 from CFP class, interview)

Learners also reported contrasting experiences regarding the availability of support when encountering difficulties. Many learners in the SDP class reported that the teacher did not provide timely assistance because many were competing to ask questions. In contrast, most learners ($N=20$) from the CFP class believed they could receive timely assistance through the CodewithAI platform:

I often encounter code-related issues in class, but the teachers cannot respond promptly. (S1 from the SDP class, interview)
ChatGPT has been beneficial for my programming learning.....When I encountered unfamiliar code, it provided reminders and explained the meaning of the code quickly. (S15 from the CFP class, reflection text).

Discussion and Implications

Addressing research questions

This study employed a quasi-experimental design and combined platform log files, summary data, and interview data, along with various analytical methods, including lag-sequential analysis, statistical analysis, and thematic analysis, to compare the effects of SDP and CFP learning modes on programming learning among Chinese primary school students. Specifically, it compared the impact of the two learning modes on novice learners' programming, including knowledge gains, skill improvement, and attitude changes. Moreover, it examined learners' behavioral patterns in the CFP class.

Learners' programming knowledge and skills

The research findings revealed differences between the two learning modes. Programming knowledge performance differed significantly between the SDP and CFP classes, suggesting that access to ChatGPT-facilitated learning may influence how primary learners acquire programming concepts. This result is consistent with prior research showing that students who use ChatGPT in programming contexts demonstrate improved learning outcomes (Hu et al., 2023). One possible explanation,

supported by our qualitative findings, is that ChatGPT provides fine-grained, context-aware feedback that supports the construction of conceptual understanding, which is particularly beneficial for novice programmers (Yang et al., 2024). Beyond offering corrective information, ChatGPT also appeared to lower learners' cognitive load by interpreting error messages, clarifying code structures, and suggesting next steps (Ye et al., 2025). This type of responsive scaffolding may help primary learners develop a coherent understanding of programming logic, which in turn can strengthen their short-term performance and accelerate the acquisition of procedural knowledge (Wang et al., 2025).

However, our findings showed that primary students' computational thinking skills did not differ significantly between the CFP and SDP classes, contrary to the results reported by Liao et al. (2024). Although the quantitative results did not indicate a significant improvement in CTS, the qualitative accounts suggest that some students nevertheless perceived gains in their problem-solving abilities through engaging with programming tasks. This discrepancy highlights the distinction between measurable CTS outcomes and primary learners' subjective experiences. Several factors may account for this result. First, the relatively short duration of the intervention may have prevented primary learners from having enough time to internalize the CTS development (Sun et al., 2023). Second, the CTS measurement instrument may have lacked sensitivity, making it difficult to capture subtle changes in young learners' thinking (Guss et al., 2024). Third, the behavioral data indicated that primary learners frequently relied on ChatGPT to explain console outputs (ECO). When explaining error messages from console outputs, ChatGPT often provided not only clarifications but also direct solutions. For example, when primary students clicked the "Explain Console Output" button, ChatGPT responded, *"SyntaxError indicates a syntax issue, and 'invalid character' means an illegal character is present. The error occurs on line 12; please try adding the correct number after 'else' to ensure the code is complete."* Although such guidance helped primary students resolve coding errors quickly and supported their acquisition of programming knowledge, it may have undermined their independent thinking, which is essential for tackling complex problems (Ribeiro et al., 2023). In particular, when primary students encountered difficulties with algorithmic design, debugging, or iterative refinement, their frequent reliance on ChatGPT reduced opportunities for problem-solving, potentially limiting gains in CTS (Shute et al., 2017).

Learners' programming behaviors

Analysis of CodewithAI interaction logs indicated that primary learners frequently executed code and, when

encountering error messages, relied primarily on the platform's error and code explanations. This high frequency suggests that the immediate, ChatGPT-generated feedback played a central role in supporting their task progression. These observations are consistent with previous research indicating that the abstract nature of programming concepts makes both comprehension and application challenging for younger learners (Kwon, 2017; Shi et al., 2018). Similar to college students' preferences (Ghimire and Edwards 2024; Sun et al. 2024a; Yilmaz and Yilmaz 2023), primary school students also tended to utilize ChatGPT to explain code and understand error messages in the console window, highlighting the importance of designing agentic AI that can provide concise, just-in-time debugging support. Furthermore, data collected from the platform revealed that the frequency of free-form questioning was the lowest. This may be due to the limited cognitive development of primary learners, despite their status as digital natives, as well as their limited experience with computers, challenges with typing, and fewer opportunities to engage in open-ended conversations with ChatGPT. As several primary students noted in their reflections, they hoped for a voice-interaction feature to reduce the need for typing. These findings point to concrete design implications for the CodewithAI platform's core functionalities to better support younger learners.

Moreover, in terms of programming behavior patterns, primary learners often engaged in free-form questioning (RoO) after requesting that ChatGPT explain the code (ECC). This sequence suggests that learners actively attempted to refine their understanding by seeking additional information through multiple rounds of interactions with ChatGPT. As also observed by Yang et al. (2025), such iterative questioning supports the gradual construction of programming knowledge. From a design perspective, this implies that agentic AI for younger learners should encourage multi-step clarification by offering follow-up prompts and opportunities for learners to ask additional questions. Integrating ChatGPT into the learning environment of younger learners addresses the limitations of traditional intelligent tutoring systems, which typically rely on predefined, rule-based algorithms (Yang et al., 2024). By leveraging large language models to generate coherent, contextually relevant responses, ChatGPT provides personalized support and real-time feedback that help primary learners stay engaged and deepen their understanding of programming concepts.

In this study, the CodewithAI platform adhered to these design principles in a form usable by primary learners. By combining a structured programming task list, an embedded Python environment, and a constrained ChatGPT interface with preset scaffolding functions (e.g., explaining console output or current code),

the CodewithAI platform channels LLM-based support toward productive programming activities. Prior research has emphasized that programming learning tools are most effective when they provide adaptive feedback aligned with learners' needs and simulate tutor- or peer-like interactions (Yang et al., 2025). In this regard, the platform demonstrates how agentic AI can be tailored to the cognitive and motivational needs of younger learners and offers actionable guidance for future LLM-based learning environments.

Learners' programming attitudes

The study compared changes in primary learners' attitudes towards programming under the two learning modes. The results showed significant differences in the confidence dimension between the two learning modes. However, no significant difference was found in the enjoyment and future interest dimensions, which is consistent with previous studies (Sun et al., 2021). The results can be explained through Bandura's (2010) theory of self-efficacy: learners with high self-efficacy are more likely to choose challenging tasks and persist longer in the face of difficulties, whereas those with lower self-efficacy are more likely to give up. For younger learners, confidence and self-efficacy are particularly important in shaping early programming experiences (Chen et al., 2025). Learners at this age often have limited exposure to abstract reasoning and may perceive programming as highly challenging (Kwon, 2017; Shi et al., 2018). When learners feel confident, they are more willing to experiment, persist through errors, and engage meaningfully in debugging, behaviors that are essential for developing foundational programming competence (Wang et al., 2025).

In this study, primary learners in the CFP class exhibited relatively high confidence compared with those in the SDP class. This research finding is consistent with the study by Del Olmo-Muñoz et al. (2020). This could be attributed to primary learners receiving not only assistance from instructors and peers but also support from ChatGPT, which enables them to overcome difficulties promptly and increases their confidence and self-efficacy. The qualitative findings further corroborate this interpretation: more than half of the learners reported that receiving timely responses from ChatGPT helped them complete tasks more efficiently, suggesting that ChatGPT exhibited human-like qualities similar to those of an instructor or peer (Xu et al., 2025). Previous studies have shown that timely learning feedback is positively correlated with students' self-efficacy (Hu, 2024); this immediacy of support may have contributed to their increased confidence. In contrast, primary learners in the SDP class were more likely to give up when they encountered coding difficulties and did not receive timely

assistance, thereby reinforcing lower levels of confidence and self-efficacy.

Although the increase in confidence represents a meaningful motivational benefit, the absence of significant gains in CTS warrants careful consideration. Consistent with research indicating that motivational factors influence programming achievement in complex ways (Nannim & Mosia, 2026), improvements in confidence may not immediately translate into measurable skill gains. These findings suggest a tension in the use of agentic AI for primary programming education. Immediate AI support may facilitate task completion, yet it may also reduce opportunities for sustained independent problem-solving (Liao et al., 2024). However, the development of CTS typically requires iterative debugging, strategic decomposition, and engagement in productive struggle (Xu et al., 2026). When explanations and solutions are readily available, learners may complete tasks efficiently without fully internalizing underlying problem-solving processes. For young learners, effective integration of agentic AI may therefore depend on adjusting support to maintain confidence while preserving opportunities for independent reasoning and skill development (Ma et al., 2025).

Furthermore, primary learners in the CFP class did not differ significantly from those in the SDP class in terms of "enjoyment" and "future interest". One possible explanation is that confidence can be strengthened relatively quickly through successful experiences (Yilmaz & Yilmaz, 2023). Whereas interest and future intentions, as more complex and stable motivational constructs, generally require a more extended period and richer positive experiences to develop. Consequently, short-term interventions may not produce rapid improvements in these dimensions (Sun et al., 2022). Qualitative data also supported these quantitative findings. Only a small number of primary learners expressed interest in continuing to study programming. Their reluctance appeared to stem from challenges encountered during the learning process, the lack of timely assistance when solving problems, and occasional issues with the responsiveness of the CodewithAI platform. These findings further highlight the distinction between learners' short-term confidence and the more enduring enjoyment and long-term interest.

Consistent with previous research, this study found that, compared with the self-directed programming learning mode, the ChatGPT-facilitated programming learning mode may enhance primary learners' programming knowledge, skills, and attitudes (Sun et al. 2024a). By incorporating advanced AI technologies, such as ChatGPT, into K-12 programming education, this research provided empirical evidence and practical implications for leveraging generative AI to support programming learning.

Implications

This study integrated ChatGPT into primary school students' programming education. Compared with the SDP learning mode, the CFP learning mode can enhance learners' programming knowledge to some extent, providing a reference for instructors in programming education. First, instructors can leverage LLM capabilities, such as ChatGPT, to improve the quality and confidence of learners' programming in the future. In the CFP class, instructors may allow learners to engage in 20 min of programming practice during which they can receive assistance and feedback from ChatGPT. In contrast, traditional classrooms typically rely on the instructor for feedback, which can delay help-seeking. This lack of timely assistance can reduce learners' confidence and interest in programming. Therefore, instructors can use ChatGPT as a personalized assistant for learners' programming instruction in the future (Surameery & Shakor, 2023). However, to enhance learners' independent thinking abilities, it is essential for primary students to use ChatGPT appropriately rather than excessively (Yang et al., 2024). For example, instructors may establish usage limits—such as a maximum number of ChatGPT queries per lesson—to reserve support for critical needs and promote initial independent effort.

Second, educators should provide scaffolding to support primary learners' use of ChatGPT. Results indicated that learners executed code more frequently than they engaged in free-form Q&A with ChatGPT. This phenomenon was closely related to the fact that primary learners were encountering this emerging AI technology for the first time. Learners still face challenges in understanding how to communicate effectively with ChatGPT. Therefore, educators should provide the necessary support and resources in the classroom. This includes demonstrating to learners how to engage effectively with ChatGPT and how to design prompts, helping primary learners better grasp programming concepts. Additionally, educators should help primary learners interpret ChatGPT's responses. Instructors can facilitate this by engaging in dialogue with ChatGPT during class and interpreting its feedback, thereby enabling primary learners to understand and adjust their code accordingly (Sun et al. 2024a). Moreover, by demonstrating the interaction process with ChatGPT, instructors can encourage students to critically evaluate the accuracy and feasibility of the feedback provided. Such instructional practices may also help students recognize the potential for AI-generated inaccuracies (e.g., hallucinations) and discourage over-reliance on ChatGPT's responses (Sun et al. 2024b).

Third, the CodewithAI platform developed in this study was well received by primary learners, providing developers with technical support for integrating ChatGPT into programming education. Interviews indicated

that primary learners found the CodewithAI platform user-friendly and genuinely helpful in their programming skills. However, some learners have suggested that, when introducing programming tasks, the font size should be increased and key content highlighted in different colors. This may be because of the limited reading ability of primary school students. They can quickly grasp the core issues when the focus is marked in the platform. Therefore, future agentic AI platforms should lower the interaction burden for novices (e.g., through multi-modal or voice input) and provide streamlined, age-appropriate explanations that make complex error messages more accessible (Ma et al., 2025). In addition, agentic AI systems could be designed to integrate large language models with Socratic questioning strategies (Xi et al., 2025), incorporating heuristic and reflective prompts that may deepen learners' cognitive engagement and promote the development of higher-order thinking skills. Such AI system design should be tailored to primary learners' habits and focus on developing human-centered AI tools (Yilmaz & Yilmaz, 2023).

Conclusion, limitations, and future directions

Programming education has become a compulsory course in K-12 education, offering significant advantages for developing learners' higher-order thinking skills, such as computational thinking and problem-solving ability. The ChatGPT-facilitated learning mode is both feasible and user-friendly in the context of programming education. This study employed a mixed-methods approach to compare two learning modes in Chinese primary education - Self-Directed Programming (SDP) and ChatGPT-Facilitated Programming (CFP) - by examining learners' programming knowledge, skills, and attitudes. It also analyzed learners' programming behavior patterns in the CFP class. The results revealed differences between the two learning modes in knowledge gains and changes in attitudes toward programming, but no significant differences in skill improvement. Primary learners in the CFP class were likely to seek help through ChatGPT. In addition to the findings, the CodewithAI platform developed in this study is another contribution of this research. Designers of future AI-supported programming environments may build on the principles of CodewithAI to balance accessibility, timely support, and the development of independent problem-solving skills. The results provide valuable insights for programming education, teacher instruction, and the design of generative AI tools. Specifically, this study provides practical guidance for instructors on using intelligent technologies in early-age programming education, supports primary learners in ChatGPT-facilitated programming learning, and offers recommendations to companies and other institutions for designing and developing generative AI tools suitable

for primary learners. The emergence of AI tools such as ChatGPT presents both significant opportunities and new challenges for programming education. This study broadens new learning strategies in primary programming education.

This study has several limitations. First, due to environmental constraints, behavioral data were not collected in the SDP class. Future studies should collect behavioral data from the control class using screen and video recordings, and compare them with those from the experimental class to better understand potential differences in learner behavior. Second, the relatively short duration (ten weeks) may introduce bias and limit the ability to capture longer-term learning trends. Future research should extend the observation period to examine the sustainability of learning outcomes. Furthermore, the sample size (seventy-one participants) was relatively small. Given the considerable variability in primary learners' learning abilities, this may limit the statistical power of the findings and affect the generalizability of the results to broader populations (Rashed Ibraheam Almoresh, 2024). Future studies should recruit larger, more diverse samples to validate the findings. Finally, the CodewithAI platform is an initial attempt. We will address issues such as font size and key-point highlighting to improve user engagement and retention in the future. Although this study was conducted in the specific context of China, our findings have broader significance for global scholars, policymakers, and educators. It is necessary to conduct further academic research to explore the potential impact of artificial intelligence LLMs on the programming learning of primary school students. Making ChatGPT and similar platforms powerful aids for primary school students in learning programming.

Appendix A

Attitudes survey.

1. I am skilled at programming.
2. I can perform well in programming classes.
3. I enjoy programming.
4. I am interested in programming.
5. I plan to take more programming courses in the future.

Appendix B

Interview.

1. Has your interest in programming changed after the course?
2. What is the most impressive part of you after the programming course?
3. In what ways do you think ChatGPT/website has assisted you?

4. What abilities have you improved after this course?
5. Do you plan to continue learning programming in the future?

Acknowledgements

The authors would like to thank all the participants for their active participation and collaboration in this study. We would also like to thank the editor and reviewers for their constructive comments that helped us revise this article.

Author contributions

Jie Xu designed and facilitated this research, analyzed the data, and wrote the first draft of the manuscript; Dan Sun facilitated data analysis and revised the manuscript; Yue Feng completed the instruction work in this research and collected data; Rustam Shadiev revised the manuscript; and Yan Li built connections with the experimental school and proofread the manuscript. All authors read and approved the final manuscript.

Funding

This work was supported by funding from the National Key Research and Development Program of China, [Grant No. 2022ZD0115904] and the National Natural Science Foundation of China [Grant No.62307011].

Data availability

The datasets used and analyzed during the current study available from the corresponding author on reasonable request.

Declarations

Ethics approval and consent to participate

This study was approved by the institutional review board of the School of Psychological and Behavioral Science at Zhejiang University ([2023] No. 070), and approval was granted on October 23, 2023. Participation was voluntary and anonymous.

Informed consent

Informed consent was gathered from all participating students. Confidentiality was maintained by not requesting names or any other information that would identify the students involved. The subjects were informed of their right to withdraw from the investigation at any time.

Competing interests

The authors declare no competing interests.

Received: 14 August 2025 / Accepted: 27 February 2026

Published online: 16 March 2026

References

- Bandura, A. (2010). Self-Efficacy. In The Corsini Encyclopedia of Psychology (eds I.B. Weiner and W.E. Craighead). <https://doi.org/10.1002/9780470479216.corpsy0836>
- Becker, B. A., Glanville, G., Iwashima, R., McDonnell, C., Goslin, K., & Mooney, C. (2016). Effective compiler error message enhancement for novice programming students. *Computer Science Education*, 26(2–3), 148–175. <https://doi.org/10.1080/08993408.2016.1225464>
- Braun, V., & Clarke, V. (2006). Using thematic analysis in psychology. *Qualitative Research in Psychology*, 3(2), 77–101.
- Bringula, R. (2024). ChatGPT in a programming course: benefits and limitations. In *Frontiers in Education*. Frontiers Media SA, 9, 1248705. <https://doi.org/10.3389/educ.2024.1248705>
- Chen, C. M., & Huang, M. Y. (2024). Enhancing programming learning performance through a Jigsaw collaborative learning method in a metaverse virtual space. *International Journal of STEM Education*, 11(1), 36. <https://doi.org/10.1186/s40594-024-00495-2>
- Chen, B., Resendes, M., Chai, C. S., & Hong, H. Y. (2017). Two tales of time: uncovering the significance of sequential patterns among contribution types in knowledge-building discourse. *Interactive Learning Environments*, 25(2), 162–175. <https://doi.org/10.1080/10494820.2016.1276081>

- Chen, S. J., Shan, X., Liu, Z. M., & Chen, C. Q. (2025). Employing large language models to enhance K-12 students' programming debugging skills, computational thinking, and self-efficacy. *Educational Technology & Society*, 28, 2.
- Cheng, M., Lai, X., Chiu, T. K., Feng, Y., & Sun, D. (2025). Unveiling children's computational thinking developmental trajectories: profiles and contextual influences in programming education. *Education and Information Technologies*, 30(18), 26631–26660. <https://doi.org/10.1007/s10639-025-13763-7>
- Cruz García, I., Martín García, J. A., Pérez Marín, D., & Pizarro, C. (2021). A proposal of programming didactics in primary education following a gamified approach with educational videogames. *Education in the knowledge society: EKS*. <https://doi.org/10.14201/eks.26130>
- del Olmo-Muñoz, J., Cózar-Gutiérrez, R., & González-Calero, J. A. (2020). Computational thinking through unplugged activities in early years of Primary Education. *Computers & Education*, 150, 103832. <https://doi.org/10.1016/j.compedu.2020.103832>
- Demirkiran, M. C., & Tansu Hocanin, F. (2021). An investigation on primary school students' dispositions towards programming with game-based learning. *Education and Information Technologies*, 26(4), 3871–3892. <https://doi.org/10.1007/s10639-021-10430-5>
- Doleck, T., Bazelaïs, P., Lemay, D. J., Saxena, A., & Basnet, R. B. (2017). Algorithmic thinking, cooperativity, creativity, critical thinking, and problem solving: Exploring the relationship between computational thinking skills and academic performance. *Journal of Computers in Education*, 4(4), 355–369.
- Duncan, C., Bell, T., & Tanimoto, S. (2014). Should your 8-year-old learn coding? In Proceedings of the 9th workshop in primary and secondary computing education (pp. 60–69). <https://www.codemonkey.com/blog/should-your-8-year-old-learn-coding/>
- Dunder, N., Lundborg, S., Wong, J., & Viberg, O. (2024). Kattis vs ChatGPT: Assessment and evaluation of programming tasks in the age of artificial intelligence. In Proceedings of the 14th Learning Analytics and Knowledge Conference (pp. 821–827).
- Fan, G., Liu, D., Zhang, R., & Pan, L. (2025). The impact of AI-assisted pair programming on student motivation, programming anxiety, collaborative learning, and programming performance: A comparative study with traditional pair programming and individual approaches. *International Journal of STEM Education*, 12(1), 16. <https://doi.org/10.1186/s40594-025-00537-3>
- Gao, Z., Yan, H., Liu, J., Zhang, X., Lin, Y., Zhang, Y., & Feng, J. (2025). Tracing distinct learning trajectories in introductory programming course: a sequence analysis of score, engagement, and code metrics for novice computer science vs. math cohorts. *International Journal of STEM Education*, 12(1), 27. <https://doi.org/10.1186/s40594-025-00546-2>
- Ghimire, A., & Edwards, J. (2024). Coding with AI: How Are Tools Like ChatGPT Being Used by Students in Foundational Programming Courses. In A. M. Olney, I. A. Chounta, Z. Liu, O. C. Santos, & I. I. Bittencourt (Eds.), *Artificial Intelligence in Education. AIED 2024. Lecture Notes in Computer Science* (Vol. 14830). Springer. https://doi.org/10.1007/978-3-031-64299-9_20
- Glesne, C. (2016). *Becoming qualitative researchers: An introduction*. Pearson.
- Gomes, A., Ke, W., Lm, S. K., Siu, A., Mendes, A. J., & Marcelino, M. J. (2017). A teacher's view about introductory programming teaching and learning—Portuguese and Macanese perspectives. In *2017 IEEE Frontiers in Education Conference (FIE)* (pp. 1–8). IEEE.
- Grover, S., Jackiw, N., & Lundh, P. (2019). Concepts before coding: Non-programming interactives to advance learning of introductory programming concepts in middle school. *Computer Science Education*, 29(2–3), 106–135. <https://doi.org/10.1080/08993408.2019.1568955>
- Guest, G., MacQueen, K., & Namey, E. (2012). *Applied thematic analysis*. Sage.
- Guner, H., Er, E., Akcapinar, G., & Khalil, M. (2024). From chalkboards to AI-powered learning. *Educational Technology & Society*, 27(2), 386–404. <https://www.jstor.org/stable/48766180>
- Guss, S. S., Clements, D. H., Sharifnia, E., Sarama, J., Holland, A., Lim, C. I., & Vinh, M. (2024). Designing Inclusive Computational Thinking Learning Trajectories for the Youngest Learners. *Education Sciences*, 14(7), 733. <https://doi.org/10.3390/educsci14070733>
- Haindl, P., & Weinberger, G. (2024). Students' experiences of using ChatGPT in an undergraduate programming course. *IEEE Access: Practical Innovations, Open Solutions*, 12, 43519–43529. <https://doi.org/10.1109/ACCESS.2024.3380909>
- Han, A., & Cai, Z. (2023). Design implications of generative AI systems for visual storytelling for young learners. In *Proceedings of the 22nd annual ACM interaction design and children conference* (pp. 470–474).
- Heintz, F., Mannila, L., & Färnqvist, T. (2016). A review of models for introducing computational thinking, computer science and computing in K–12 education. *Teoksessa 2016 IEEE frontiers in education conference (FIE)*. (pp. 1–9). IEEE.
- Hopcan, S., Polat, E., & Albayrak, E. (2022). Collaborative behavior patterns of students in programming instruction. *Journal of Educational Computing Research*, 60(4), 1035–1062. <https://doi.org/10.1177/07356331211062260>
- Hou, H. T., & Wang, S. M. (2015). Analyzing students' cognitive processing patterns of a socioscientific issue learning activity with online discussion from a process perspective- results of quantitative content analysis and lag sequential analysis. *International Institute of Applied Informatics*, 1(3), 49e58. <https://doi.org/10.1109/IIAI-AAI.2015.216>
- Hu, Y. H. (2024). Implementing generative AI chatbots as a decision aid for enhanced values clarification exercises in online business ethics education. *Educational Technology & Society*, 27(3), 356–373. <https://www.jstor.org/stable/48787035>
- Hu, M., Assadi, T., & Mahrooian, H. (2023). Explicitly introducing ChatGPT into first-year programming practice: Challenges and Impact. In *2023 IEEE International Conference on Teaching, Assessment and Learning for Engineering (TALE)* (pp. 1–6). IEEE.
- Iskrenovic-Momcilovic, O. (2019). Pair programming with scratch. *Education and Information Technologies*, 24, 2943–2952. <https://doi.org/10.1007/s10639-019-09905-3>
- Jing, Y., Wang, H., Chen, X., & Wang, C. (2024). What factors will affect the effectiveness of using ChatGPT to solve programming problems? A quasi-experimental study. *Humanities and Social Sciences Communications*, 11(1), 1–12. <https://doi.org/10.1057/s41599-024-02751-w>
- Kang, C., Liu, N., Zhu, Y., Li, F., & Zeng, P. (2023). Developing College students' computational thinking multidimensional test based on Life Story situations. *Educational and Information Technologies*, 28, 2661–2679. <https://doi.org/10.1007/s10639-022-11189-z>
- Khor, E. T., & Chan, L. (2025, June). Exploring the effect of scaffolding strategies in GenAI chatbot on student engagement and programming skill development. In *The Global Chinese Conference on Computers in Education (GCCCE) Proceedings* (Vol. 2025, pp. 32–39).
- Khor, E. T., Chan, L., Koh, E., & Seow, P. (2025). Exploring students' perceptions of generative AI-generated feedback in learning programming. *Educational Psychology*. <https://doi.org/10.1080/01443410.2025.2564894>
- Kong, S. C., & Wang, Y. Q. (2021). Investigating primary school principals' programming perception and support from the perspective of reasoned action: A mixed methods approach. *Computers & Education*, 172, 104267. <https://doi.org/10.1016/j.compedu.2021.104267>
- Kroustalli, C., & Xinogalos, S. (2021). Studying the effects of teaching programming to lower secondary school students with a serious game: a case study with Python and CodeCombat. *Education and Information Technologies*, 26(5), 6069–6095. <https://doi.org/10.1007/s10639-021-10596-y>
- Kwon, K. (2017). Novice programmer's misconception of programming reflected on problem-solving plans. *International Journal of Computer Science Education in Schools*, 1(4), 14–24. <https://doi.org/10.21585/ijcses.v1i4.19>
- Liao, J., Zhong, L., Zhe, L., Xu, H., Liu, M., & Xie, T. (2024). Scaffolding computational thinking with ChatGPT. *IEEE Transactions on Learning Technologies*, 17, 1668–1682. <https://doi.org/10.1109/TLT.2024.3392896>
- Lindberg, R. S., Laine, T. H., & Haaranen, L. (2019). Gamifying programming education in K-12: A review of programming curricula in seven countries and programming games. *British journal of educational technology*, 50(4), 1979–1995. <https://doi.org/10.1111/bjet.12685>
- Liu, P. L., & Chen, C. J. (2023). Using an AI-Based Object Detection Translation Application for English Vocabulary Learning. *Educational Technology & Society*, 26(3), 5–20. [https://doi.org/10.30191/ETS.202307_26\(3\).0002](https://doi.org/10.30191/ETS.202307_26(3).0002)
- Lye, S. Y., & Koh, J. H. L. (2014). Review on teaching and learning of computational thinking through programming: What is next for K-12? *Computers in human behavior*, 41, 51–61. <https://doi.org/10.1016/j.chb.2014.09.012>
- Ma, X., Li, C., Xu, J., Zhu, S., & Li, Y. (2025). Exploring characteristics of primary school students' self-regulated learning (SRL) behaviors in human-GenAI collaborative programming learning environments: Insights from a proposed framework. *Computers & Education*, 105453. <https://doi.org/10.1016/j.compedu.2025.105453>
- Manorat, P., Tuarob, S., & Pongpaichet, S. (2025). Artificial intelligence in computer programming education: A systematic literature review. *Computers and Education: Artificial Intelligence*, 100403. <https://doi.org/10.1016/j.caeai.2025.100403>
- Michel-Villareal, R., Vilalta-Perdomo, E., Salinas-Navarro, D. E., Thierry-Aguilera, R., & Gerardou, F. S. (2023). Challenges and opportunities of generative AI for higher education as explained by ChatGPT. *Education Sciences*, 13(9), 856. <https://doi.org/10.3390/educsci13090856>

- Ministry of Education (2022). *The Curriculum Standards of Information Science and Technology (IST) for Compulsory Education (2022 Edition)*. Ministry of Education of the People's Republic of China website. http://www.moe.gov.cn/srcsite/A2/6/s8001/202204/t20220420_619921.html
- Mousavinasab, E., Zarifsanaiy, N., Niakan Kalhori, R., Rakhshan, S., Keikha, M., L., & Ghazi Saeedi, M. (2021). Intelligent tutoring systems: a systematic review of characteristics, applications, and evaluation methods. *Interactive Learning Environments*, 29(1), 142–163. <https://doi.org/10.1080/10494820.2018.1558257>
- Munaye, Y. Y., Admass, W., Belayneh, Y., Molla, A., & Asmare, M. (2025). ChatGPT in Education: A Systematic Review on Opportunities, Challenges, and Future Directions. *Algorithms*, 18(6), 352. <https://doi.org/10.3390/a18060352>
- Nannim, F. A., & Mosia, M. (2026). Gender as a Moderator in the Relationship Between Teamwork, Grit, and Students' Achievement in Robotics Programming. *Journal for STEM Education Research*, 1–21. <https://doi.org/10.1007/s41979-025-00165-0>
- Neutens, T., Barbion, E., Coolsaet, K., & Wyffels, F. (2021). Comparing learning ecologies of primary graphical programming: create or fix? *Journal of computer assisted learning*, 37(5), 1296–1311. <https://doi.org/10.1111/jcal.12570>
- Ou, Q., Liang, W., He, Z., Liu, X., Yang, R., & Wu, X. (2023). Investigation and analysis of the current situation of programming education in primary and secondary schools. *Heliyon*, 9(4). <https://doi.org/10.1016/j.heliyon.2023.e15530>
- Ouyang, F., Dai, X., & Chen, S. (2022). Applying multimodal learning analytics to examine the immediate and delayed effects of instructor scaffoldings on small groups' collaborative programming. *International Journal of STEM Education*, 9(1), 1–21. <https://doi.org/10.1186/s40594-022-00361-z>
- Ouyang, F., Guo, M., Zhang, N., Bai, X., & Jiao, P. (2024). Comparing the effects of instructor manual feedback and ChatGPT intelligent feedback on collaborative programming in China's higher education. *IEEE Transactions on Learning Technologies*, 17, 2173–2185. <https://doi.org/10.1109/TLT.2024.3486749>
- Papert, S. (1991). Situating constructionism. In I. Harel & S. Papert (Eds.), *Constructionism: Research reports and essays* (pp. 1–11). Norwood.
- Pereira, F. D., Oliveira, E. H., Oliveira, D. B., Cristea, A. I., Carvalho, L. S., Fonseca, S. C., & Isotani, S. (2020). Using learning analytics in the Amazonas: understanding students' behaviour in introductory programming. *British journal of educational technology*, 51(4), 955–972. <https://doi.org/10.1111/bjet.12953>
- Piedade, J., Dorotea, N., Pedro, A., & Matos, J. F. (2020). On teaching programming fundamentals and computational thinking with educational robotics: A didactic experience with pre-service teachers. *Education Sciences*, 10(9), 214. <https://doi.org/10.3390/educsci10090214>
- Rahman, M. M., & Watanobe, Y. (2023). ChatGPT for education and research: Opportunities, threats, and strategies. *Applied Sciences*, 13(9), 5783. <https://doi.org/10.3390/app13095783>
- Rashed Ibraheem Almohesh, A. (2024). AI Application (ChatGPT) and Saudi Arabian Primary School Students' Autonomy in Online Classes: Exploring Students and Teachers' Perceptions. *The International Review of Research in Open and Distributed Learning*, 25(3), 1–18. <https://doi.org/10.19173/irrodl.v25i3.7641>
- Rekha, K., Gopal, K., Satheeskumar, D., Anand, U. A., Doss, D. S. S., & Elayaperumal, S. (2024, May). AI-powered personalized learning system design: Student engagement and performance tracking system. In *2024 4th International Conference on Advance Computing and Innovative Technologies in Engineering (ICACITE)* (pp. 1125–1130). IEEE.
- Ribeiro, J. L., Mattos, G. O., Araújo, T. S., & Study in schools of the municipality of alagoa grande in the state of paraíba/brazil. (2023). the potential impact of artificial intelligence chatbots in basic EDUCATION: A. In ICERI2023 Proceedings (pp. 9336–9340). IATED. <https://doi.org/10.21125/iceri.2023.2395>
- Sentance, S., & Csizmadia, A. (2017). Computing in the curriculum: Challenges and strategies from a teacher's perspective. *Education and Information Technologies*, 22(2), 469–495. <https://doi.org/10.1007/s10639-016-9482-0>
- Shi, N., Cui, W., Zhang, P., & Sun, X. (2018). Evaluating the effectiveness roles of variables in the novice programmers learning. *Journal of Educational Computing Research*, 56(2), 181–201. <https://doi.org/10.1177/0735633117707312>
- Shoufan, A. (2023). Exploring students' perceptions of ChatGPT: Thematic analysis and follow-up survey. *Ieee Access: Practical Innovations, Open Solutions*, 11, 38805–38818. <https://doi.org/10.1109/ACCESS.2023.3268224>
- Shute, V. J., Sun, C., & Asbell-Clarke, J. (2017). Demystifying computational thinking. *Educational research review*, 22, 142–158. <https://doi.org/10.1016/j.edurev.2017.09.003>
- Sisman, B., Kucuk, S., & Ozcan, N. (2022). Collaborative behavioural patterns of elementary school students working on a robotics project. *Journal of Computer Assisted Learning*, 38(4), 1018–1032. <https://doi.org/10.1111/jcal.12659>
- Sun, D., Ouyang, F., Li, Y., & Zhu, C. (2021). Comparing learners' knowledge, behaviors, and attitudes between two instructional modes of computer programming in secondary education. *International Journal of STEM Education*, 8, 1–15. <https://doi.org/10.1186/s40594-021-00311-1>
- Sun, L., Hu, L., & Zhou, D. (2022). Programming attitudes predict computational thinking: Analysis of differences in gender and programming experience. *Computers & Education*, 181, 104457. <https://doi.org/10.1016/j.compedu.2022.104457>
- Sun, D., Zhu, C., Xu, F., Li, Y., Ouyang, F., & Cheng, M. (2023). Transitioning From Introductory to Professional Programming in Secondary Education: Comparing Learners' Computational Thinking Skills, Behaviors, and Attitudes. *Journal of Educational Computing Research*, 62(3), 427–454. <https://doi.org/10.1177/07356331231204653>
- Sun, D., Boudouaia, A., Zhu, C., & Li, Y. (2024a). Would ChatGPT-facilitated programming mode impact college students' programming behaviors, performances, and perceptions? An empirical study. *International Journal of Educational Technology in Higher Education*, 21(1), 14–22. <https://doi.org/10.1186/s41239-024-00446-5>
- Sun, D., Boudouaia, A., Yang, J., & Xu, J. (2024b). Investigating students' programming behaviors, interaction qualities and perceptions through prompt-based learning in ChatGPT. *Humanities and Social Sciences Communications*, 11(1447), 1–14. <https://doi.org/10.1057/s41599-024-03991-6>
- Sun, L., Liu, J., & Liu, Y. (2024c). Comparative experiment of the effects of unplugged and plugged-in programming on computational thinking in primary school students: A perspective of multiple influential factors. *Thinking Skills and Creativity*, 52, 101542. <https://doi.org/10.1016/j.tsc.2024.101542>
- Surameery, N. M. S., & Shakor, M. Y. (2023). Use chatgpt to solve programming bugs. *International Journal of Information Technology and Computer Engineering*, 3, 17–22. <https://doi.org/10.55529/ijitc.31.17.22>
- Tan, J., Wu, L., & Ma, S. (2024). Collaborative dialogue patterns of pair programming and their impact on programming self-efficacy and coding performance. *British Journal of Educational Technology*, 55(3), 1060–1081. <https://doi.org/10.1111/bjet.13412>
- Tobias, R. G., Lozano, J. A. G., Torres, M. L. M., Ramírez, J. A., Baldini, G. M., & Okoye, K. (2025). AI and VR integration for enhancing ethical decision-making skills and competency of learners in higher education. *International Journal of STEM Education*, 12(1), 52. <https://doi.org/10.1186/s40594-025-00575-x>
- Topali, P., & Mikropoulos, T. A. (2021). Scratch-based learning objects for novice programmers: exploring quality aspects and perceptions for primary education. *Interactive Learning Environments*, 31(7), 4219–4234. <https://doi.org/10.1080/10494820.2021.1956546>
- Tsai, M. J., Liang, J. C., & Hsu, C. Y. (2021). The computational thinking scale for computer literacy education. *Journal of Educational Computing Research*, 59(4), 579–602. <https://doi.org/10.1177/0735633120972356>
- Uğraş, H., Uğraş, M., Papadakis, S., & Kalogiannakis, M. (2024). ChatGPT-Supported Education in Primary Schools: The Potential of ChatGPT for Sustainable Practices. *Sustainability*, 16(22), 9855. <https://doi.org/10.3390/su16229855>
- Vujinović, A., Luburić, N., Slivka, J., & Kovačević, A. (2024). Using ChatGPT to annotate a dataset: A case study in intelligent tutoring systems. *Machine Learning with Applications*, 16, 100557. <https://doi.org/10.1016/j.mlwa.2024.100557>
- Wang, M., Wang, M., Xu, X., Yang, L., Cai, D., & Yin, M. (2023). Unleashing ChatGPT's Power: A Case Study on Optimizing Information Retrieval in Flipped Classrooms via Prompt Engineering. *IEEE Transactions on Learning Technologies*, 8, 1–13. <https://doi.org/10.1109/TLT.2023.3324714>
- Wang, Z., Zou, D., Zhang, R., Lee, L. K., Xie, H., & Wang, F. L. (2025). ChatGPT-enhanced self-regulated learning in programming education: impacts on motivation, self-efficacy, and learning outcomes. *Interactive Learning Environments*. <https://doi.org/10.1080/10494820.2025.2559919>
- Wong, G. K. (2024). Amplifying children's computational problem-solving skills: A hybrid-based design for programming education. *Education and Information Technologies*, 29(2), 1761–1793. <https://doi.org/10.1007/s10639-023-11880-9>
- Wong, G. K. W., & Cheung, H. Y. (2020). Exploring children's perceptions of developing twenty-first century skills through computational thinking and programming. *Interactive Learning Environments*, 28(4), 438–450. <https://doi.org/10.1080/10494820.2018.1534245>
- Wu, B., Hu, Y., Ruis, A. R., & Wang, M. (2019). Analysing computational thinking in collaborative programming: A quantitative ethnography approach. *Journal of Computer Assisted Learning*, 35(3), 421–434. <https://doi.org/10.1111/jcal.12348>
- Xi, L., Zhang, Y., & Wang, Q. (2025). Investigating the effects of an LLM-based Socratic conversational agent on students' academic performance and reflective thinking in higher education. *Computers & Education*. <https://doi.org/10.1016/j.compedu.2025.105494>

- Xu, J., Li, Y., Shadiev, R., & Li, C. (2025). College students' use behavior of generative AI and its influencing factors under the unified theory of acceptance and use of technology model. *Education and Information Technologies*. <https://doi.org/10.1007/s10639-025-13508-6>
- Xu, J., Chen, Z., Chen, M., Li, Y., & Xu, X. (2026). Unpacking the Effects of GenAI on Cultivating Students' Computational Thinking: A Meta-Analysis. *Journal of Educational Computing Research*. <https://doi.org/10.1177/07356331261419586>
- Yang, A. C., Lin, J. Y., Lin, C. Y., & Ogata, H. (2024). Enhancing python learning with PyTutor: Efficacy of a ChatGPT-Based intelligent tutoring system in programming education. *Computers and Education: Artificial Intelligence*, 7, 100309. <https://doi.org/10.1016/j.caeai.2024.100309>
- Yang, T. C., Hsu, Y. C., & Wu, J. Y. (2025). The effectiveness of ChatGPT in assisting high school students in programming learning: evidence from a quasi-experimental research. *Interactive Learning Environments*. <https://doi.org/10.1080/10494820.2025.2450659>
- Ye, X., Zhang, W., Zhou, Y., Li, X., & Zhou, Q. (2025). Improving students' programming performance: an integrated mind mapping and generative AI chatbot learning approach. *Humanities and Social Sciences Communications*, 12(1), 1–13. <https://doi.org/10.1057/s41599-025-04846-4>
- Yilmaz, R., & Yilmaz, F. G. K. (2023). Augmented intelligence in programming learning: Examining student views on the use of ChatGPT for programming learning. *Computers in Human Behavior: Artificial Humans*, 1(2), 100005. <https://doi.org/10.1016/j.chbah.2023.100005>

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.